

Lecture 09

SQL Access with Dapper

Continuation of the layered Expense Tracker

Dapper is explicit SQL with light object mapping.

- Replace JSON repositories with Dapper and SQLite.
- Keep Application contracts unchanged.
- Use parameters for every SQL value.
- Ownership stays in `WHERE Id = @id AND UserId = @userId`.

Domain: User and Expense.

Application: repository interfaces and validation.

Infrastructure: connections, SQL, Dapper mapping.

Presentation: console conversion.

Project: composition root.

`SELECT` maps rows to C# objects.

`INSERT` uses parameters, never concatenated input.

`DELETE` requires both expense id and current user id.

Database constraints back up service validation.

- Read the runnable CodeExamples project first.
- Run the Project and identify where each responsibility lives.
- Change one implementation detail without breaking the Application layer.
- Write a short explanation of the boundary you protected.

- Apply the same decision to the student's chosen domain.
- Keep a screenshot or short note for the evolution portfolio.
- Use the database-session model and constraints.
- Prepare one demonstration scenario for the final defense.